

Tugas Besar 2

IF3270 Pembelajaran Mesin

CNN dan RNN



Disusun oleh:

Ivant Samuel Silaban	13523129
Muhamad Nazih Najmudin	13523144
Muhammad Rizain Firdaus	13523164

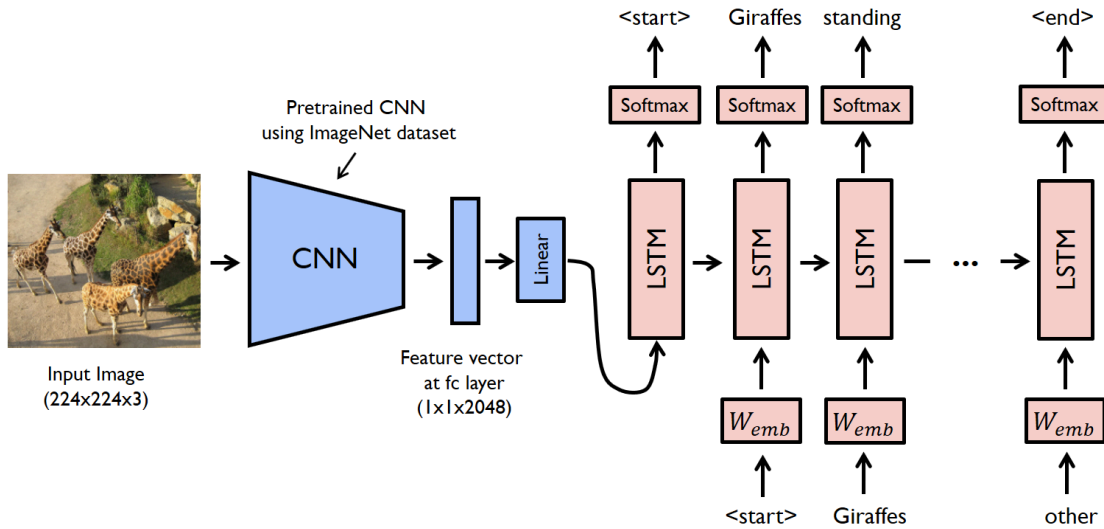
**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI.....	1
BAB I: Deskripsi Persoalan.....	2
BAB II: Pembahasan.....	3
2.1 Implementasi.....	3
2.1.1 Deskripsi Kelas, Atribut, dan Metodenya.....	4
2.1.1.1 CNN.....	4
1. Kelas Conv2D Layer (conv2d.py).....	4
2. Kelas LocallyConnected2DLayer (locally_connected.py).....	5
3. Kelas Pooling (pooling.py).....	5
4. Kelas FlattenLayer (flatten.py).....	6
5. Kelas Arsitektur (model.py).....	6
6. Kelas Util (utils.py).....	6
2.1.1.2 RNN.....	7
1. Kelas DecoderRNN.....	7
2. Kelas SimpleRNN.....	8
2.1.1.3 LSTM.....	8
1. Kelas DecoderLSTM.....	8
2. Kelas LSTM.....	9
2.1.2 Forward Propagation.....	10
2.1.2.1 CNN.....	10
1. Lapisan Konvolusi (Convolutional Layers)	
Lapisan konvolusi mengekstraksi fitur lokal dengan melakukan sliding window kernel	
dalam melintasi dimensi spasial peta fitur input.....	10
2.1.2.2 RNN.....	13
1. Projection Layer.....	13
2. Embedding Layer.....	14
3. Recurrent Layer.....	15
4. Dense Output Layer.....	15
2.1.2.3 LSTM.....	16
2.2 Hasil Pengujian.....	17
2.2.1 CNN.....	17
2.2.1.1 Pengaruh Jumlah Layer Konvolusi.....	19
2.2.1.2 Pengaruh Banyak Filter per Layer Konvolusi.....	22
2.2.1.3 Pengaruh Ukuran Filter per Layer Konvolusi.....	25
2.2.1.4 Pengaruh Jenis Pooling Layer.....	28
2.2.1.5 Analisis LocallyConnected2D Scratch (Shared vs Non-Shared).....	31
2.2.1.6 Analisis Conv2D Scratch.....	33

2.2.2 RNN.....	35
2.2.2.1 Pengaruh Jumlah Layer.....	35
2.2.2.2 Pengaruh Hidden Size.....	37
2.2.2.3 Perbandingan Keras.....	38
2.2.3 LSTM.....	40
2.2.3.1 Pengaruh Jumlah Layer.....	40
2.2.3.2 Pengaruh Hidden Size.....	41
2.2.3.3 Perbandingan Keras.....	42
2.2.4 RNN & LSTM.....	44
2.2.5 Pengaruh Max Caption Length.....	45
BAB III: Kesimpulan dan Saran.....	47
3.1 Kesimpulan.....	47
3.2 Saran.....	47
LAMPIRAN.....	48
REFERENSI.....	49
Lampiran Form Penggunaan AI.....	50

BAB I: Deskripsi Persoalan



Gambar 1.1 Arsitektur Encoder-Decoder CNN dan RNN/LSTM

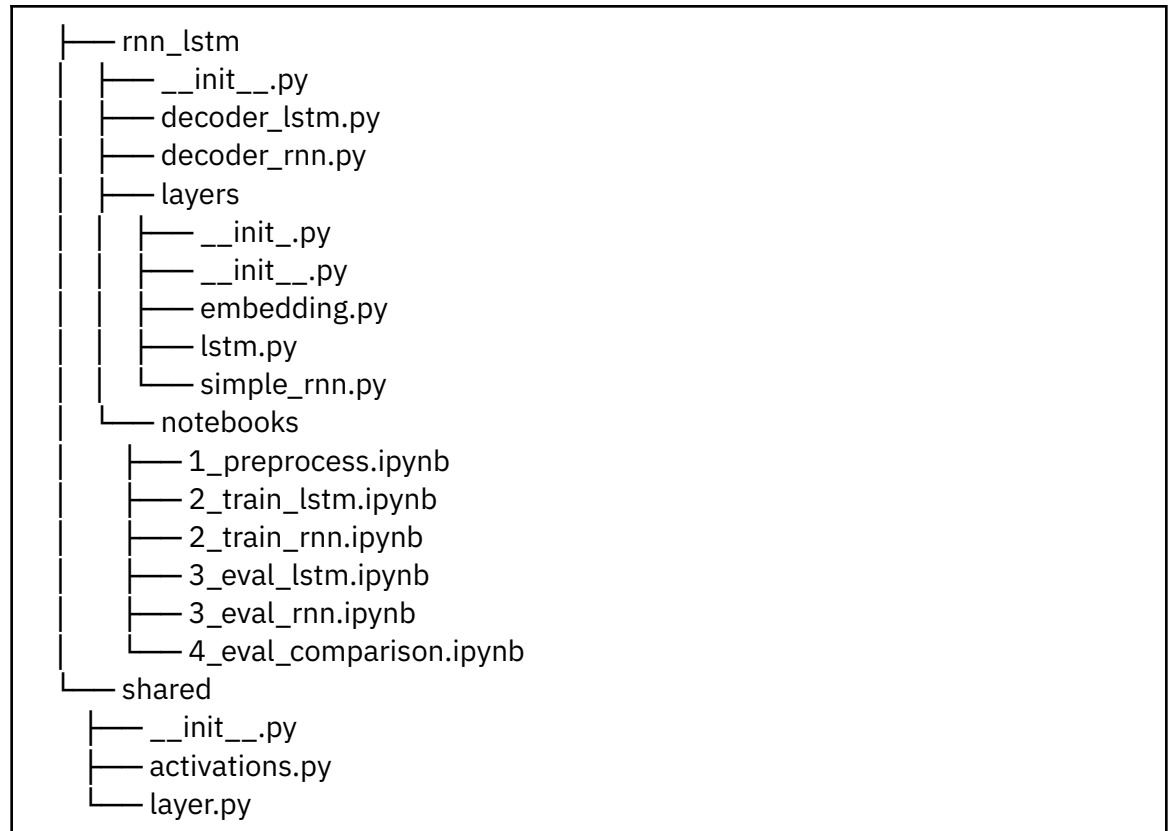
Tugas Besar II pada kuliah IF3270 Pembelajaran Mesin bertujuan agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan modul forward propagation CNN, Simple RNN, dan LSTM from scratch. Selain itu, peserta juga akan mengeksplorasi pipeline image captioning melalui arsitektur encoder-decoder yang menggabungkan CNN dan LSTM, yang merupakan fondasi dari banyak sistem deep learning modern seperti LLM multimodal dan speech recognition.

BAB II: Pembahasan

2.1 Implementasi

Tabel 2.1.1 Struktur direktori program

```
CNN-RNN-LSTM-Scratch
├── .gitignore
├── README.md
├── doc
├── requirements.txt
├── src
│   ├── __init__.py
│   ├── cnn
│   │   ├── __init__.py
│   │   ├── layers
│   │   │   ├── __init__.py
│   │   │   ├── conv2d.py
│   │   │   ├── flatten.py
│   │   │   ├── locally_connected.py
│   │   │   └── pooling.py
│   │   ├── model.py
│   │   └── notebooks
│   │       ├── layer-1-32-3x3-avgpool.ipynb
│   │       ├── layer-1-32-3x3-maxpool.ipynb
│   │       ├── layer-1-32-5x5-avgpool.ipynb
│   │       ├── layer-1-32-5x5-maxpool.ipynb
│   │       ├── layer-1-64-3x3-avgpool.ipynb
│   │       ├── layer-1-64-3x3-maxpool.ipynb
│   │       ├── layer-1-64-5x5-avgpool.ipynb
│   │       ├── layer-1-64-5x5-maxpool.ipynb
│   │       ├── layer-2-32-64-3x3-avgpool.ipynb
│   │       ├── layer-2-32-64-3x3-maxpool.ipynb
│   │       ├── layer-2-32-64-5x5-avgpool.ipynb
│   │       ├── layer-2-32-64-5x5-maxpool.ipynb
│   │       ├── layer-2-64-128-3x3-avgpool.ipynb
│   │       ├── layer-2-64-128-3x3-maxpool.ipynb
│   │       ├── layer-2-64-128-5x5-avgpool.ipynb
│   │       ├── layer-2-64-128-5x5-maxpool.ipynb
│   │       ├── locally_connected_scratch.ipynb
│   │       └── scratch_vs_keras.ipynb
│   └── utils.py
└── data
```



2.1.1 Deskripsi Kelas, Atribut, dan Metodenya

2.1.1.1 CNN

1. Kelas Conv2D Layer (conv2d.py)

Kelas ini mengimplementasikan lapisan konvolusi 2D dengan *shared parameters*) yang mendukung ekstraksi fitur spasial.

- Atribut:

- kernel: Tensor bobot filter dengan dimensi (kH, kW, C_{in}, C_{out})
- bias: vektor bias untuk setiap filter C_{out}
- stride: besaran pergeseran filter selama operasi konvolusi
- padding: strategi penanganan tepi gambar
- activation: nama fungsi aktivasi yang diterapkan setelah operasi konvolusi.

- Metode:

- forward(x): melakukan operasi konvolusi menggunakan np.einsum untuk kontraksi *tensor* yang efisien antara *window* input dan *kernel*.

- `_get_pad(H, W)`: fungsi helper untuk menghitung jumlah padding yang dibutuhkan.
- `_apply_activation(x)`: menerapkan fungsi aktivasi non-linear pada hasil akumulasi konvolusi

2. Kelas **LocallyConnected2DLayer** (**locally_connected.py**)

Mengimplementasikan lapisan konvolusi tanpa pembagian parameter (non-shared parameters), di mana setiap lokasi spasial memiliki filter uniknya sendiri.

- Atribut:
 - `kernel`: Tensor bobot filter dengan dimensi (kH, kW, C_in, C_out)
 - `bias`: vektor bias untuk setiap filter C_out
 - `stride`: besaran pergeseran filter selama operasi konvolusi
 - `padding`: strategi penanganan tepi gambar.
 - `activation`: nama fungsi aktivasi yang diterapkan setelah operasi konvolusi.
 - `kH, kW`: dimensi tinggi dan lebar kernel
- Metode:
 - `forward(x)`: melakukan operasi konvolusi menggunakan `np.einsum` untuk kontraksi *tensor* yang efisien antara *window* input dan *kernel*.
 - `_get_pad(H, W)`: fungsi helper untuk menghitung jumlah padding yang dibutuhkan.
 - `_apply_activation(x)`: menerapkan fungsi aktivasi non-linear pada hasil akumulasi konvolusi

3. Kelas **Pooling** (**pooling.py**)

Modul ini berisi kelas-kelas untuk operasi reduksi dimensi spasial melalui teknik pooling.

- Kelas **MaxPooling2DLayer** / **AveragePooling2DLayer**:
 - Atribut: `pool_h`, `pool_w` (dimensi jendela), `stride_h`, `stride_w` (langkah pergeseran).
 - Metode: `forward(x)` menerapkan operasi statistik maksimum atau rata-rata pada setiap kanal secara independen menggunakan *sliding window*.
- Kelas **GlobalAveragePooling2DLayer** / **GlobalMaxPooling2DLayer**:
 - Metode: `forward(x)` mereduksi seluruh peta fitur spasial H x W menjadi satu nilai representatif per kanal melalui

operasi rata-rata atau maksimum global.

4. Kelas FlattenLayer (flatten.py)

Bertanggung jawab untuk transformasi tensor multidimensi menjadi vektor linear.

- Atribut: `input_shape` untuk menyimpan dimensi asli input sebelum proses perataan.
- Metode: `forward(input)` melakukan perataan tensor menggunakan tatanan memori row-major (C-order) agar kompatibel dengan Dense layer.

5. Kelas Arsitektur (model.py)

Komponen integrasi utama yang merakit seluruh lapisan menjadi satu model utuh.

- Kelas CNNModel:
 - Atribut: `layers` (daftar objek lapisan yang menyusun arsitektur).
 - Metode:
 - `_build_from_keras(keras_model)`: Factory method yang mem-parsing objek model Keras dan mengkonversinya menjadi urutan lapisan from scratch serta memindahkan bobotnya.
 - `forward(x)`: Mengalirkan data melalui setiap lapisan secara berurutan.
 - `predict(x)`: Menghasilkan prediksi kelas akhir berdasarkan probabilitas tertinggi.
- Kelas ActivationLayer:
 - Atribut: `activation_name` (string identifikasi) dan objek `activations`.
 - Metode: `forward(x)` sebagai pembungkus (wrapper) untuk memanggil fungsi aktivasi spesifik dari modul [activations.py](#).

6. Kelas Util (utils.py)

Kumpulan fungsi helper untuk mengelola *pipeline* pre-processing data.

- Fungsi `load_image`:
 - atribut: `path` (jalur ke lokasi image), `target_size` (dimensi spasial target output)
- `batch_load_images`:

- atribut: paths (kumpulan string jalur direktori images), target_size (dimensi spasial target output)
- extract_features:
 - atribut:
 - paths: kumpulan string jalur direktori images
 - encoder: ekstraktor vektor fitur spasial
 - target_size: Ukuran dimensi resolusi input
 - output_dir: Jalur folder/direktori lokal tempat penyimpanan berkas biner hasil ekstraksi fitur.
 - batch_size: Jumlah kuantitas gambar yang dikelompokkan untuk diproses secara paralel oleh encoder dalam satu kali siklus forward pass.

2.1.1.2 RNN

1. Kelas DecoderRNN

Kelas ini bertindak sebagai assembler utama yang mengimplementasikan arsitektur forward propagation secara utuh. Kelas ini mengintegrasikan seluruh komponen komputasi seperti Embedding, Dense, dan SimpleRNN, serta mengatur aliran pemrosesan data menggunakan mekanisme pre-inject untuk menggabungkan modalitas visual dan tekstual.

- **Atribut:**
 - **num_layers:** Bilangan bulat yang menentukan jumlah lapisan RNN yang digunakan dalam arsitektur.
 - **activation:** Instans kelas Activation yang digunakan untuk memanggil fungsi-fungsi aktivasi seperti ReLU dan Softmax.
 - **embedding:** Instans kelas Embedding yang berfungsi mengubah indeks token teks menjadi representasi vektor.
 - **projection_layer:** Instans kelas Dense yang bertugas menyusutkan dimensi fitur visual CNN agar ekuivalen dengan dimensi word embedding.
 - **rnn_layers:** List yang menampung instans kelas SimpleRNN.
 - **output_layer:** Instans kelas Dense yang berfungsi memetakan hidden state dari RNN menjadi nilai logits sebesar jumlah total kosakata.
- **Metode:**

- **set_model_params(self, params):** Mengaplikasikan matriks bobot hasil ekstraksi model Keras.
- **forward(self, image_features, caption_tokens):** Modul eksekusi utama yang menjalankan forward propagation.

2. Kelas SimpleRNN

Kelas ini mengimplementasikan komputasi matematis dari sel RNN yang dirancang untuk menyimpan, memproses, dan memperbarui status hidden state menggunakan operasi kombinasi linear dan aktivasi tanh.

- **Atribut:**

- **input_dim:** Ukuran dimensi vektor fitur input yang diterima pada setiap timestep.
- **hidden_size:** Ukuran kapasitas memori atau dimensi output hidden state.
- **W_xh:** Matriks bobot masukan berdimensi (input_dim, hidden_size) untuk mentransformasi vektor fitur data yang masuk.
- **W_hh:** Matriks bobot recurrent berdimensi (hidden_size, hidden_size) untuk mentransformasi hidden state dari timestep sebelumnya.
- **b_h:** Vektor bias berdimensi (hidden_size,) yang ditambahkan pada perhitungan linear.

- **Metode:**

- **set_weights(self, keras_weights):** Menerima bobot dari Keras dan mengaplikasikannya ke matriks dalam kelas.
- **forward(self, inputs):** Melakukan forward propagation untuk sebuah sel RNN.

2.1.1.3 LSTM

1. Kelas DecoderLSTM

Kelas ini bertindak sebagai assembler utama yang mengimplementasikan arsitektur forward propagation secara utuh. Kelas ini mengintegrasikan seluruh komponen komputasi seperti Embedding, Dense, dan LSTM, serta mengatur aliran pemrosesan data menggunakan mekanisme pre-inject untuk menggabungkan modalitas visual dan tekstual.

- **Atribut:**

- **num_layers:** Bilangan bulat yang menentukan jumlah lapisan LSTM yang digunakan dalam arsitektur.

- **activation:** Instans kelas Activation yang digunakan untuk memanggil fungsi-fungsi aktivasi seperti ReLU dan Softmax.
- **embedding:** Instans kelas Embedding yang berfungsi mengubah indeks token teks menjadi representasi vektor.
- **projection_layer:** Instans kelas Dense yang bertugas menyusutkan dimensi fitur visual CNN agar ekuivalen dengan dimensi word embedding.
- **lstm_layers:** List yang menampung instans kelas SimpleRNN.
- **output_layer:** Instans kelas Dense yang berfungsi memetakan hidden state dari LSTM menjadi nilai logits sebesar jumlah total kosakata.
- **Metode:**
 - **set_model_params(self, params):** Mengaplikasikan matriks bobot hasil ekstraksi model Keras.
 - **forward(self, image_features, caption_tokens):** Modul eksekusi utama yang menjalankan forward propagation.

2. Kelas LSTM

Kelas ini mengimplementasikan komputasi matematis dari sel LSTM yang dirancang untuk menyimpan, memproses, dan memperbarui status hidden state menggunakan operasi kombinasi linear dan aktivasi tanh.

- **Atribut:**
 - **input_dim:** Ukuran dimensi vektor fitur input yang diterima pada setiap timestep.
 - **hidden_size:** Ukuran kapasitas memori cell state dan dimensi output hidden state.
 - **activation:** Instans kelas Activation yang digunakan untuk memanggil fungsi-fungsi aktivasi seperti Sigmoid.
 - **W_x:** Matriks bobot input berdimensi (input_dim, hidden_size * 4) untuk mentransformasi vektor fitur data yang masuk.
 - **W_h:** Matriks bobot recurrent berdimensi (hidden_size, hidden_size * 4) untuk mentransformasi hidden state dari timestep sebelumnya.
 - **b:** Vektor bias berukuran (hidden_size * 4,) sebagai offset untuk keempat control gate.
- **Metode:**

- **set_weights(self, keras_weights):** Menerima bobot dari Keras dan mengaplikasikannya ke matriks dalam kelas.
- **forward(self, inputs):** Melakukan forward propagation untuk sebuah sel LSTM.

2.1.2 Forward Propagation

Proses forward propagation dalam Convolutional Neural Network (CNN) merupakan serangkaian transformasi tensor secara berurutan. Transformasi ini memetakan ruang input gambar awal (piksel spasial) ke dalam ruang representasi fitur tingkat tinggi, yang pada akhirnya didistribusikan menjadi probabilitas kelas diskrit. Implementasi from scratch mengandalkan aljabar linear dan operasi matriks menggunakan NumPy untuk mereplikasi TensorFlow/Keras.

2.1.2.1 CNN

1. Lapisan Konvolusi (Convolutional Layers)

Lapisan konvolusi mengekstraksi fitur lokal dengan melakukan *sliding window kernel* dalam melintasi dimensi spasial peta fitur input.

a. Shared Parameters (Conv2DLayer)

Pada konvolusi standar, bobot kernel yang sama diaplikasikan di seluruh lokasi spasial. Secara komputasional, ini adalah penjumlahan dari *element-wise dot product*. Untuk mengoptimalkan komputasi dan meminimalisir iterasi *loop*, implementasi menggunakan notasi konvensi Einstein (`np.einsum`) untuk melakukan kontraksi *tensor* orde-3 dengan *tensor* orde 3 secara langsung.

Berikut adalah potongan kode dari `src/cnn/layers/conv2d.py`:

```
def forward(self, x: np.ndarray) -> np.ndarray:
    # TODO: Implement forward pass
    # pass
    H, W, C_in = x.shape
    assert C_in == self.C_in

    # pad_h, pad_w = self.kH // 2, self.kW // 2
    pad_h, pad_w = self._get_pad(H, W)
    if pad_h > 0 or pad_w > 0:
        x = np.pad(x, ((pad_h, pad_h), (pad_w, pad_w), (0, 0)), mode="constant")
        H, W = x.shape[:2]

    out_H = (H - self.kH) // self.stride + 1
    out_W = (W - self.kW) // self.stride + 1
    out = np.zeros((out_H, out_W, self.C_out))

    for i in range(out_H):
        for j in range(out_W):
            patch = x[i*self.stride : i * self.stride+self.kH,
                      j*self.stride : j * self.stride+self.kW, :]
            out[i, j, :] = np.einsum("hwc, hwk -> k", patch, self.kernel) + self.bias

    return self._apply_activation(out)
```

b. Non-Shared Parameters (LocallyConnected2DLayer)

Berbeda dengan konvolusi standar, lapisan ini tidak memiliki *translational invariance*. Setiap koordinat spasial output (i,j) memiliki himpunan parameter filternya sendiri yang diindeks melalui `pos_idx`. Karena vektorisasi *einsum* tidak lagi berlaku linear, *patch* spasial perlu diratakan menggunakan `ravel()` sebelum dikalikan dengan parameter posisi spesifik.

Berikut adalah potongan kode dari `src/cnn/layers/locally_connected.py`:

```
def forward(self, x: np.ndarray) -> np.ndarray:
    H, W, C_in = x.shape

    pad_h, pad_w = self._get_pad(H, W)
    if pad_h > 0 or pad_w > 0:
        x = np.pad(x, ((pad_h, pad_h), (pad_w, pad_w), (0, 0)), mode="constant")
        H, W = x.shape[:2]

    out_H = (H - self.kH) // self.stride + 1
    out_W = (W - self.kW) // self.stride + 1

    expected_output_pixels = out_H * out_W
    assert self.kernel.shape[0] == expected_output_pixels, \
        f"Mismatch: Kernel memiliki {self.kernel.shape[0]} posisi, tetapi output butuh {expected_output_pixels}"

    out = np.zeros((out_H, out_W, self.kernel.shape[-1]))

    for i in range(out_H):
        for j in range(out_W):
            pos_idx = i * out_W + j

            patch = x[i*self.stride : i*self.stride + self.kH,
                      j*self.stride : j*self.stride + self.kW, :]

            patch_flat = patch.ravel()

            out[i, j, :] = np.dot(patch_flat, self.kernel[pos_idx]) + self.bias[pos_idx]

    return self._apply_activation(out)
```

2. Lapisan Pooling

Tujuan dari lapisan ini adalah untuk mencapai *spatial invariance* dengan mengurangi resolusi dari *feature maps*. Setiap *pooled feature map* berkorespondensi dengan *feature map* di layer sebelumnya. Unit-unitnya menggabungkan input dari patch kecil $n \times n$ unit. *Pooling window* ini memiliki ukuran yang beragam dan operasi di *window* ini dapat dengan mencari nilai maksimum (MaxPool) atau nilai rata-rata (AveragePool) di *window tersebut*.

Berikut adalah implementasi dari lapisan pooling yang dapat dilihat di `src/cnn/layers/pooling.py`:

```

class MaxPooling2DLayer:
    def __init__(self, keras_layer):
        cfg = keras_layer.get_config()
        self.pool_h, self.pool_w = cfg["pool_size"]
        self.stride_h, self.stride_w = cfg['strides']

    def forward(self, x: np.ndarray) -> np.ndarray:
        H, W, C = x.shape
        out_H = (H - self.pool_h) // self.stride_h + 1
        out_W = (W - self.pool_w) // self.stride_w + 1
        out = np.zeros((out_H, out_W, C), dtype=np.float32)

        for i in range(out_H):
            for j in range(out_W):
                window = x[i*self.stride_h : i * self.stride_h+self.pool_h,
                           j*self.stride_w : j * self.stride_w+self.pool_w, :]
                out[i, j, :] = np.max(window, axis=(0, 1))

        return out

class AveragePooling2DLayer:
    def __init__(self, keras_layer):
        cfg = keras_layer.get_config()
        self.pool_h, self.pool_w = cfg["pool_size"]
        self.stride_h, self.stride_w = cfg['strides']

    def forward(self, x: np.ndarray) -> np.ndarray:
        H, W, C = x.shape
        out_H = (H - self.pool_h) // self.stride_h + 1
        out_W = (W - self.pool_w) // self.stride_w + 1
        out = np.zeros((out_H, out_W, C), dtype=np.float32)

        for i in range(out_H):
            for j in range(out_W):
                window = x[i*self.stride_h : i*self.stride_h + self.pool_h,
                           j*self.stride_w : j*self.stride_w + self.pool_w, :]
                out[i, j, :] = np.mean(window, axis=(0, 1))

        return out

```

3. Flatten & Pipeline

Sebelum feature maps yang sudah di-pooling tersebut dapat dikomputasi di *dense* layer, feature map tersebut yang merupakan tensor multidimensi (H, W, C) harus ditransformasi menjadi vektor linear 1D (Flattening).

Berikut implementasi dari flattening di `src/cnn/layers/flatten.py`:

```

class FlattenLayer:
    def forward(self, input):
        self.input_shape = input.shape
        return input.flatten()

```

2.1.2.2 RNN

```
8 class DecoderRNN:
62 def forward(self, image_features: np.ndarray, caption_tokens: np.ndarray) -> np.ndarray:
63     """
64     Forward propagation untuk satu batch data
65     - image_features: Fitur hasil ekstraksi CNN (batch_size, 2048)
66     - caption_tokens: Indeks token caption (batch_size, seq_len)
67     """
68     # Proyeksi Fitur Gambar
69     img_x = self.projection_layer.forward(image_features)
70     img_x = self.activation.relu(img_x) # (batch_size, embed_dim)
71     img_x = np.expand_dims(img_x, axis=1) # (batch_size, 1, embed_dim)
72
73     # Word Embedding
74     word_x = self.embedding.forward(caption_tokens) # (batch_size, seq_len, embed_dim)
75
76     # Pre-Injection
77     x = np.concatenate([img_x, word_x], axis=1) # (batch_size, seq_len + 1, embed_dim)
78
79     # Pass through RNN
80 for rnn_layer in self.rnn_layers:
81     x = rnn_layer.forward(x)
82
83     # Slicing Output -> ambil hasil timestep teks
84     x_words = x[:, 1:, :] # (batch_size, seq_len, hidden_size)
85
86     # Dense & Softmax
87     logits = self.output_layer.forward(x_words)
88     probs = self.activation.softmax(logits) # (batch_size, seq_len, vocab_size)
89
90     return probs
```

Proses forward propagation pada decoder RNN dirancang menggunakan arsitektur pre-inject, di mana informasi visual dan teks digabungkan sejak awal sebelum masuk ke dalam memori recurrent. Secara garis besar, alur ini dimulai dengan memproses matriks fitur gambar dari CNN dan matriks indeks token teks secara paralel. Fitur gambar diproyeksikan ke dalam dimensi ruang yang sama dengan representasi kata, lalu disisipkan tepat di depan sekuens teks sebagai timestep awal ($t = -1$). Sekuens gabungan ini kemudian diproses dengan hidden layer RNN untuk menghasilkan riwayat hidden state. Pada tahap akhir, hidden state untuk porsi gambar di-slicing, dan sisa hidden state yang berkorespondensi dengan teks dipetakan menjadi distribusi probabilitas kosakata untuk memprediksi kata demi kata berikutnya.

1. Projection Layer

```
# Dense Projection Layer
self.projection_layer = Dense(in_features=2048, out_features=embed_dim)
```

Lapisan ini berfungsi sebagai jembatan dimensional antara output konvolusi dan decoder RNN. Pada implementasi yang dibangun, lapisan ini menggunakan instansiasi dari kelas Dense. Lapisan ini menerima fitur gambar hasil ekstraksi pre-trained model InceptionV3 yang memiliki

dimensi awal sebesar 2048, lalu mengalikannya dengan matriks bobot proyeksi untuk membuat dimensinya menjadi ekuivalen dengan ukuran word embedding. Setelah operasi linear selesai, fungsi aktivasi ReLU diaplikasikan. Matriks yang dihasilkan kemudian diekspansi menjadi bentuk tensor 3D dengan panjang sekuens 1 agar dapat diteruskan ke layer berikutnya.

2. Embedding Layer

```
3 class Embedding:
37     def forward(self, inputs: np.ndarray) -> np.ndarray:
38         """
39         Parameters:
40         - inputs: Matriks 2D berisi indeks token dengan tipe integer
41         - Shape : (batch_size, sequence_length)
42
43         Returns:
44         - np.ndarray: Tensor 3D hasil pemetaan indeks ke vektor.
45         - Shape      : (batch_size, sequence_length, embed_dim)
46         """
47         if not np.issubdtype(inputs.dtype, np.integer):
48             raise TypeError("Input untuk Embedding layer harus berupa array integer (indeks token).")
49
50         out: np.ndarray = self.weights[inputs]
51         return out
```

Berbeda dengan layer Dense yang melakukan perkalian matriks secara linear, Embedding layer yang diimplementasikan pada tugas besar ini hanya beroperasi sebagai tabel pencarian (seperti lookup table). Layer ini menerima batch yang berisi indeks bilangan bulat (integer) dari token-token kata. Layer ini mengambil baris vektor berukuran embed_dim dari dalam matriks bobotnya (self.weights) yang bersesuaian dengan setiap indeks input. Hasil akhirnya adalah transformasi ke bentuk tensor 3D yang merepresentasikan urutan kata.

3. Recurrent Layer

```
4 class SimpleRNN:
8     def forward(self, inputs: np.ndarray) -> np.ndarray:
9         """
10         Parameters:
11         - inputs: Tensor 3D dari embedding layer
12         - Shape : (batch_size, sequence_length, input_dim)
13
14         Returns:
15         - np.ndarray: Tensor 3D hidden state untuk seluruh timestep
16         - Shape : (batch_size, sequence_length, hidden_size)
17         """
18         batch_size, seq_len, current_input_dim = inputs.shape
19
20         if current_input_dim != self.input_dim:
21             raise ValueError(f"Input dim tidak cocok. Ekspektasi {self.input_dim}, dapat {current_input_dim}")
22
23         # Inisialisasi h_0 dan array Output
24         h_t = np.zeros((batch_size, self.hidden_size), dtype=np.float32)
25         outputs = np.zeros((batch_size, seq_len, self.hidden_size), dtype=np.float32)
26
27         for t in range(seq_len):
28             x_t = inputs[:, t, :] # Shape: (batch_size, input_dim)
29
30             # Calculate: (x_t * W_xh) + (h_{t-1} * W_hh) + b_h
31             linear_combination = np.dot(x_t, self.W_xh) + np.dot(h_t, self.W_hh) + self.b_h
32
33             # Activation
34             h_t = np.tanh(linear_combination)
35
36             # Output h_t
37             outputs[:, t, :] = h_t
38
39         return outputs
```

Layer ini merupakan lapisan komputasi utama RNN yang bertanggung jawab untuk memproses sekuens gabungan (gambar di awal, disusul teks) melintasi dimensi waktu. Implementasi ini melakukan iterasi sepanjang sequence length. Pada setiap timestep t , lapisan RNN menghitung hidden state saat ini (h_t) menggunakan kombinasi linear dari input saat ini (x_t) dan memori timestep sebelumnya (h_{t-1}). Secara matematis, layer ini menjalankan operasi $h_t = \tanh(x_t * W_{xh} + h_{t-1} * W_{hh} + b_h)$. Karena rancangan model ini dikonfigurasi untuk mengembalikan sekuens penuh, setiap vektor h_t yang dihasilkan akan disimpan dan disusun kembali ke dalam tensor output berdimensi 3D yang merepresentasikan seluruh jejak status.

4. Dense Output Layer

```
# Output Layer
self.output_layer = Dense(in_features=hidden_size, out_features=vocab_size)
```

Sebagai tahap kulminasi, output tensor dari RNN, setelah melalui proses slicing untuk membuang hasil komputasi timestep gambar, diteruskan ke layer Dense akhir. Layer ini beroperasi pada seluruh elemen batch dan timestep untuk memetakan representasi hidden state menjadi nilai logits yang berukuran sama dengan jumlah total kosakata ($vocab_size$). Setelah itu, fungsi aktivasi Softmax diaplikasikan yang mengonversi nilai logits ke

dalam bentuk persentase distribusi probabilitas yang valid untuk setiap posisi kata dalam sekuens. Hal ini memungkinkan implementasi logika decoding untuk memilih kata dengan probabilitas tertinggi.

2.1.2.3 LSTM

Secara garis besar, alur forward propagation pada decoder LSTM memiliki struktur yang identik dengan arsitektur RNN. Keduanya mengaplikasikan mekanisme pre-inject di mana matriks fitur gambar diproyeksikan, diekspansi dimensinya, dan disisipkan pada posisi paling awal ($t = -1$) mendahului sekuens word embedding. Sekuens gabungan ini kemudian diproses secara temporal, di-slicing pada bagian output gambarnya, dan dipetakan menjadi probabilitas tebakan kata melalui projection layer dan aktivasi Softmax. Perbedaan mendasar antara keduanya terletak pada kompleksitas arsitektur di dalam recurrent layer-nya. RNN hanya menggunakan satu operasi linear tunggal, sementara LSTM memiliki memori jangka panjang berupa cell state yang diregulasi oleh 4 control gates.

```
6 class LSTM:
48     def forward(self, inputs: np.ndarray) -> np.ndarray:
60         if current_input_dim != self.input_dim:
61             raise ValueError(f"Input dim tidak cocok. Ekspektasi {self.input_dim}, dapat {current_input_dim}")
62
63         # Inisialisasi h, C, dan output
64         h_t = np.zeros((batch_size, self.hidden_size), dtype=np.float32)
65         C_t = np.zeros((batch_size, self.hidden_size), dtype=np.float32)
66         outputs = np.zeros((batch_size, seq_len, self.hidden_size), dtype=np.float32)
67
68         h = self.hidden_size
69
70         for t in range(seq_len):
71             x_t = inputs[:, t, :]
72
73             # Calculate 4 Gates
74             z = np.dot(x_t, self.W_x) + np.dot(h_t, self.W_h) + self.b # Shape: (batch_size, 4 * hidden_size)
75
76             # Split: i, f, c, o
77             z_i = z[:, :h] # Input gate
78             z_f = z[:, h:2*h] # Forget gate
79             z_c = z[:, 2*h:3*h] # Cell candidate
80             z_o = z[:, 3*h:] # Output gate
81
82             # Activation
83             i = self.activation.sigmoid(z_i)
84             f = self.activation.sigmoid(z_f)
85             C_tilde = np.tanh(z_c)
86             o = self.activation.sigmoid(z_o)
87
88             # Update Cell State
89             C_t = f * C_t + i * C_tilde
90
91             # Update Hidden State
92             h_t = o * np.tanh(C_t)
93
94             # Output h_t
95             outputs[:, t, :] = h_t
96
97         return outputs
```

Recurrent layer LSTM beroperasi sebagai inti pemrosesan temporal pada decoder ini. Untuk meminimalisasi iterasi looping di tingkat bahasa pemrograman dan memaksimalkan efisiensi operasi matriks, bobot

komputasi dari keempat gerbang digabungkan ke dalam matriks tunggal dengan ukuran ($\text{hidden_size} * 4$). Pada setiap timestep t , model menerima input (x_t) dan hidden state sebelumnya (h_{t-1}). Keduanya dikombinasikan secara linear dalam operasi: $z = x_t * W_x + h_{t-1} * W_h + b$. Matriks z yang dihasilkan kemudian dipartisi menjadi empat segmen gates dengan peran spesifik:

1. **Input Gate (i):** Mengaplikasikan fungsi aktivasi sigmoid untuk meregulasi persentase informasi baru dari input saat ini yang diizinkan masuk ke dalam memori utama.
2. **Forget Gate (f):** Mengaplikasikan fungsi aktivasi sigmoid untuk mengevaluasi cell state sebelumnya (C_{t-1}) untuk memilah informasi historis mana yang relevan untuk dipertahankan dan mana yang harus dilupakan.
3. **Cell Candidate ($C^{\wedge}$):** Menggunakan fungsi aktivasi tanh untuk mengekstrak kandidat vektor informasi baru yang berpotensi memperkaya cell state.
4. **Output Gate (o):** Mengaplikasikan fungsi aktivasi sigmoid untuk menentukan bagian mana dari cell state yang layak keluar sebagai hidden state (h_t).

Setelah keempat gate kalkulasi tersebut terbentuk, LSTM akan memperbarui cell state melalui kombinasi linier yang memadukan pembuangan memori lama dan penambahan memori baru. Rangkaian hidden state ini disimpan pada setiap iterasi waktu dan dikembalikan sebagai tensor 3D untuk diteruskan ke lapisan output.

2.2 Hasil Pengujian

2.2.1 CNN

Berikut adalah hasil pengujian terhadap 16 arsitektur dengan variasi:

- Jumlah layer konvolusi (2 variasi)
- Banyak filter per layer konvolusi (2 variasi)
- Ukuran filter per layer konvolusi (2 variasi)
- Jenis pooling layer (2 variasi: Max vs Average)

Configuration					Macro F1-Score
Jumlah Layer	Jumlah Filter		Ukuran Filter	Jenis Pooling Layer	
1	32		3x3	Average	0.7454

1	32		3x3	Max	0.7473
1	32		5x5	Average	0.7129
1	32		5x5	Max	0.746
1	64		3x3	Average	0.74
1	64		3x3	Max	0.7346
1	64		5x5	Average	0.7323
1	64		5x5	Max	0.7699
2	32	64	3x3	Average	0.7684
2	32	64	3x3	Max	0.7809
2	32	64	5x5	Average	0.7568
2	32	64	5x5	Max	0.7849
2	64	128	3x3	Average	0.7626
2	64	128	3x3	Max	0.7765
2	64	128	5x5	Average	0.7535
2	64	128	5x5	Max	0.7674

Dan berikut adalah *summary* dari hasil eksperimen:

Configuration		Average
Jumlah Layer	1	0.74105
	2	0.768875
Jumlah Filter	32	0.7379
	64	0.7442
	32-64	0.77275
	64-128	0.765
Ukuran Filter	3x3	0.7569625
	5x5	0.7529625
Jenis Pooling	Max	0.7634375
	Average	0.7464875

Atau dapat dilihat juga di: [Hasil Pengujian CNN](#)

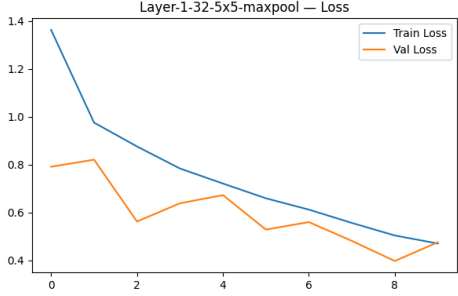
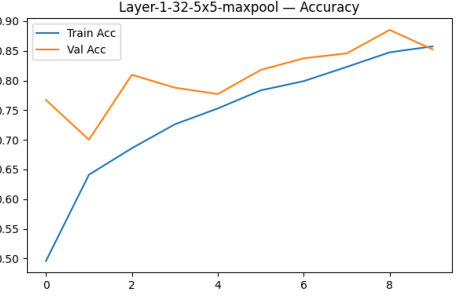
Penjelasan lebih detail dapat dilihat di bawah ini.

2.2.1.1 Pengaruh Jumlah Layer Konvolusi

Notebook Pengujian:

- **Notebook 1:** Layer-1-32-5x5-maxpool.ipynb
- **Notebook 2:** Layer-2-32-64-5x5-maxpool.ipynb

No	Hasil Pengujian																		
1	Layer-1-32-5x5-maxpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameter: Model: "Layer-1-32-5x5-maxpool" <table><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr><tr><td>conv2s_1 (Conv2D)</td><td>(None, 146, 146, 32)</td><td>2,432</td></tr><tr><td>maxpool_1 (MaxPooling2D)</td><td>(None, 73, 73, 32)</td><td>0</td></tr><tr><td>flatten (Flatten)</td><td>(None, 170528)</td><td>0</td></tr><tr><td>dense_1 (Dense)</td><td>(None, 128)</td><td>21,827,712</td></tr><tr><td>output (Dense)</td><td>(None, 6)</td><td>774</td></tr></table> Total params: 21,830,918 (83.28 MB) Trainable params: 21,830,918 (83.28 MB) Non-trainable params: 0 (0.00 B)	Layer (type)	Output Shape	Param #	conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432	maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0	flatten (Flatten)	(None, 170528)	0	dense_1 (Dense)	(None, 128)	21,827,712	output (Dense)	(None, 6)	774
Layer (type)	Output Shape	Param #																	
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432																	
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0																	
flatten (Flatten)	(None, 170528)	0																	
dense_1 (Dense)	(None, 128)	21,827,712																	
output (Dense)	(None, 6)	774																	
	Plot:																		

	Layer-1-32-5x5-maxpool — Loss <table><caption>Layer-1-32-5x5-maxpool — Loss Data</caption><tr><th>Epoch</th><th>Train Loss</th><th>Val Loss</th></tr><tr><td>0</td><td>1.35</td><td>0.80</td></tr><tr><td>1</td><td>1.00</td><td>0.82</td></tr><tr><td>2</td><td>0.90</td><td>0.58</td></tr><tr><td>3</td><td>0.80</td><td>0.65</td></tr><tr><td>4</td><td>0.75</td><td>0.68</td></tr><tr><td>5</td><td>0.70</td><td>0.55</td></tr><tr><td>6</td><td>0.65</td><td>0.58</td></tr><tr><td>7</td><td>0.60</td><td>0.55</td></tr><tr><td>8</td><td>0.55</td><td>0.42</td></tr><tr><td>9</td><td>0.50</td><td>0.48</td></tr><tr><td>10</td><td>0.48</td><td>0.48</td></tr></table> Layer-1-32-5x5-maxpool — Accuracy <table><caption>Layer-1-32-5x5-maxpool — Accuracy Data</caption><tr><th>Epoch</th><th>Train Acc</th><th>Val Acc</th></tr><tr><td>0</td><td>0.50</td><td>0.78</td></tr><tr><td>1</td><td>0.65</td><td>0.70</td></tr><tr><td>2</td><td>0.68</td><td>0.82</td></tr><tr><td>3</td><td>0.72</td><td>0.78</td></tr><tr><td>4</td><td>0.75</td><td>0.78</td></tr><tr><td>5</td><td>0.78</td><td>0.82</td></tr><tr><td>6</td><td>0.80</td><td>0.84</td></tr><tr><td>7</td><td>0.82</td><td>0.85</td></tr><tr><td>8</td><td>0.84</td><td>0.88</td></tr><tr><td>9</td><td>0.85</td><td>0.85</td></tr><tr><td>10</td><td>0.85</td><td>0.85</td></tr></table> Macro F1 Score: 0.7460	Epoch	Train Loss	Val Loss	0	1.35	0.80	1	1.00	0.82	2	0.90	0.58	3	0.80	0.65	4	0.75	0.68	5	0.70	0.55	6	0.65	0.58	7	0.60	0.55	8	0.55	0.42	9	0.50	0.48	10	0.48	0.48	Epoch	Train Acc	Val Acc	0	0.50	0.78	1	0.65	0.70	2	0.68	0.82	3	0.72	0.78	4	0.75	0.78	5	0.78	0.82	6	0.80	0.84	7	0.82	0.85	8	0.84	0.88	9	0.85	0.85	10	0.85	0.85
Epoch	Train Loss	Val Loss																																																																							
0	1.35	0.80																																																																							
1	1.00	0.82																																																																							
2	0.90	0.58																																																																							
3	0.80	0.65																																																																							
4	0.75	0.68																																																																							
5	0.70	0.55																																																																							
6	0.65	0.58																																																																							
7	0.60	0.55																																																																							
8	0.55	0.42																																																																							
9	0.50	0.48																																																																							
10	0.48	0.48																																																																							
Epoch	Train Acc	Val Acc																																																																							
0	0.50	0.78																																																																							
1	0.65	0.70																																																																							
2	0.68	0.82																																																																							
3	0.72	0.78																																																																							
4	0.75	0.78																																																																							
5	0.78	0.82																																																																							
6	0.80	0.84																																																																							
7	0.82	0.85																																																																							
8	0.84	0.88																																																																							
9	0.85	0.85																																																																							
10	0.85	0.85																																																																							
2	Layer-2-32-64-5x5-maxpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid', name='conv2s_2'), keras.layers.MaxPooling2D((2, 2), name='maxpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameters:																																																																								

Model: "Layer-2-32-64-5x5-maxpool"

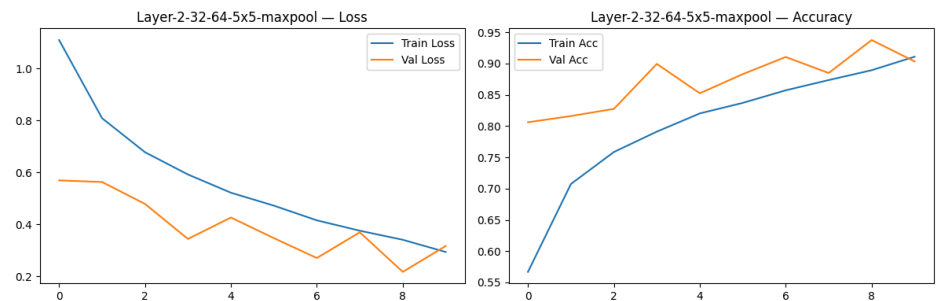
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7849

Penjelasan:

Penambahan layer konvolusi dari 1 menjadi 2 secara konsisten meningkatkan performa model. Secara teoritis, model dengan lebih banyak convolutional layer biasanya mampu menangkap pola yang lebih kompleks, terutama jika dataset memiliki struktur visual yang rumit seperti dataset ini. Dengan convolutional layer yang lebih banyak, model lebih mampu menangkap kombinasi fitur.

Dari jika dilihat dari plot visualisasi training and validation loss, dapat dilihat kalau plot visualisasi untuk model 2 layer menunjukkan penurunan *training loss* yang lebih konsisten dan mencapai titik konvergensi yang lebih rendah daripada model dengan 1 layer. Hal ini mengonfirmasi bahwa arsitektur yang lebih dalam mampu melakukan ekstraksi fitur hierarkis yang lebih kompleks. Namun, pastinya ada trade-off nya yaitu jumlah operasi komputasi yang lebih meningkat, dan risiko overfitting yang meningkat.

2.2.1.2 Pengaruh Banyak Filter per Layer Konvolusi

Notebook Pengujian:

- **Notebook 1:** Layer-2-32-64-5x5-maxpool.ipynb
- **Notebook 2:** Layer-2-64-128-5x5-maxpool.ipynb

No	Hasil Pengujian
1	Layer-2-32-64-5x5-maxpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid', name='conv2s_2'), keras.layers.MaxPooling2D((2, 2), name='maxpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameter:

Model: "Layer-2-32-64-5x5-maxpool"

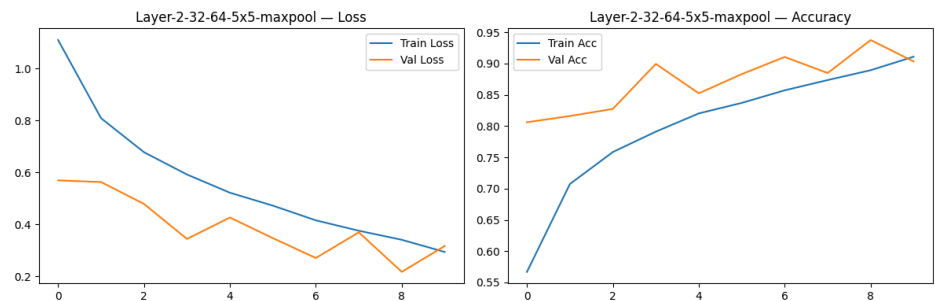
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7849

2

Layer-2-64-128-5x5-maxpool.ipynb:

```
keras.layers.Input(shape=IMG_SIZE + (3,), name='input'),
keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid',
name='conv2s_1'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_1'),
keras.layers.Conv2D(128, (5, 5), activation='relu',
padding='valid', name='conv2s_2'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_2'),
```

```
keras.layers.Flatten(name='flatten'),
keras.layers.Dense(128, activation='relu', name='dense_1'),
keras.layers.Dense(6, activation='softmax', name='output'),
```

Parameters:

Model: "Layer-2-64-128-5x5-maxpool"

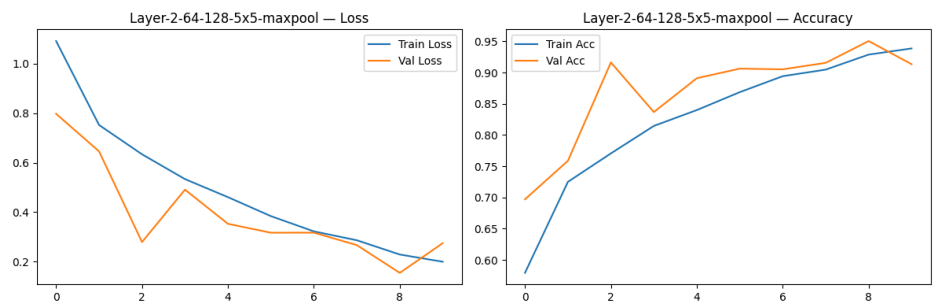
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 64)	4,864
maxpool_1 (MaxPooling2D)	(None, 73, 73, 64)	0
conv2s_2 (Conv2D)	(None, 69, 69, 128)	204,928
maxpool_2 (MaxPooling2D)	(None, 34, 34, 128)	0
flatten (Flatten)	(None, 147968)	0
dense_1 (Dense)	(None, 128)	18,940,032
output (Dense)	(None, 6)	774

Total params: 19,150,598 (73.05 MB)

Trainable params: 19,150,598 (73.05 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7674

Penjelasan:

Saat kapasitas jumlah filter tersebut ditambahkan, total parameter model meningkat hampir dua kali lipat dari 9,524,550 (model 32→64)

menjadi 19,150,598 (model 64→128). Peningkatan ini didorong terutama oleh conv layer kedua (dari 51,264 menjadi 204,928 parameter) dan dense layer pertama yang menerima flatten output lebih besar (dari 9,470,080 menjadi 18,940,032 parameter). Kapasitas model yang terlalu besar relatif terhadap dataset mengakibatkan kurva loss yang tidak stabil dan mengarah ke overfitting, yang menjadi penyebab turunnya Macro F1-Score.

2.2.1.3 Pengaruh Ukuran Filter per Layer Konvolusi

Notebook Pengujian:

- **Notebook 1:** Layer-2-32-64-3x3-maxpool.ipynb
- **Notebook 2:** Layer-2-32-64-5x5-maxpool.ipynb

No	Hasil Pengujian
1	Layer-2-32-64-3x3-maxpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (3, 3), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Conv2D(64, (3, 3), activation='relu', padding='valid', name='conv2s_2'), keras.layers.MaxPooling2D((2, 2), name='maxpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameter:

Model: "Layer-2-32-64-3x3-maxpool"

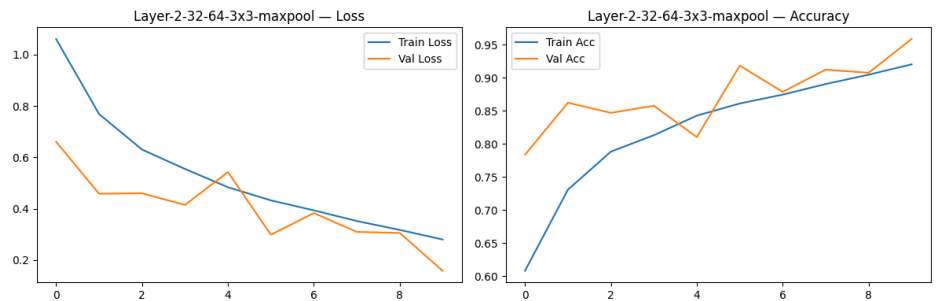
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 148, 148, 32)	896
maxpool_1 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2s_2 (Conv2D)	(None, 72, 72, 64)	18,496
maxpool_2 (MaxPooling2D)	(None, 36, 36, 64)	0
flatten (Flatten)	(None, 82944)	0
dense_1 (Dense)	(None, 128)	10,616,960
output (Dense)	(None, 6)	774

Total params: 10,637,126 (40.58 MB)

Trainable params: 10,637,126 (40.58 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7809

2

Layer-2-32-64-5x5-maxpool.ipynb:

```
keras.layers.Input(shape=IMG_SIZE + (3,), name='input'),
keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid',
name='conv2s_1'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_1'),
keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid',
name='conv2s_2'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_2'),
```

```
keras.layers.Flatten(name='flatten'),
keras.layers.Dense(128, activation='relu', name='dense_1'),
keras.layers.Dense(6, activation='softmax', name='output'),
```

Parameter:

Model: "Layer-2-32-64-5x5-maxpool"

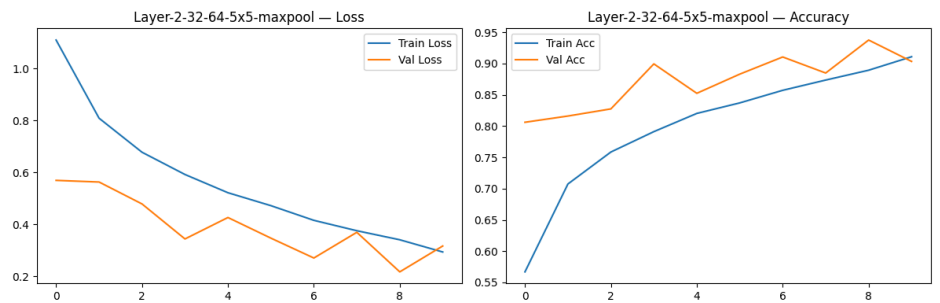
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7849

Penjelasan:

Jika dilihat dari hasil eksperimen untuk kedua notebook ini, penggunaan filter ukuran 5×5 memberikan hasil yang sedikit lebih baik (0.7849 vs 0.7809). Namun jika dilihat secara agregat dari keseluruhan 16 arsitektur, rata-rata F1 untuk kernel 3×3 (0.7570) sedikit lebih tinggi dibandingkan 5×5 (0.7530). Perbedaan ini sangat kecil (0.004 poin) sehingga secara praktis kedua ukuran kernel memberikan performa yang sebanding pada dataset ini. Hasil berbeda antara perbandingan per pasang dan agregat dapat terjadi karena efek interaksi dengan variabel lain seperti jumlah filter dan jenis pooling.

2.2.1.4 Pengaruh Jenis Pooling Layer

Notebook Pengujian:

- **Notebook 1:** Layer-2-32-64-5x5-avgpool.ipynb
- **Notebook 2:** Layer-2-32-64-5x5-maxpool.ipynb

No	Hasil Pengujian
1	Layer-2-32-64-5x5-avgpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.AveragePooling2D((2, 2), name='avgpool_1'), keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid', name='conv2s_2'), keras.layers.AveragePooling2D((2, 2), name='avgpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameter:

Model: "Layer-2-32-64-5x5-avgpool"

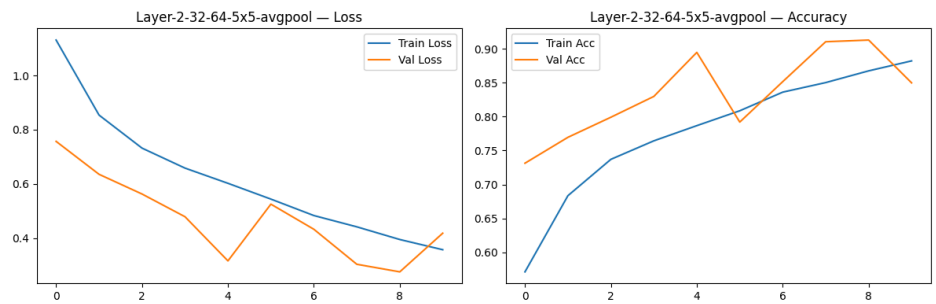
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
avgpool_1 (AveragePooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
avgpool_2 (AveragePooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7568

2

Layer-2-32-64-5x5-maxpool.ipynb:

```
keras.layers.Input(shape=IMG_SIZE + (3,), name='input'),
keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid',
name='conv2s_1'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_1'),
keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid',
name='conv2s_2'),
keras.layers.MaxPooling2D((2, 2), name='maxpool_2'),
```

```
keras.layers.Flatten(name='flatten'),
keras.layers.Dense(128, activation='relu', name='dense_1'),
keras.layers.Dense(6, activation='softmax', name='output'),
```

Parameter:

Model: "Layer-2-32-64-5x5-maxpool"

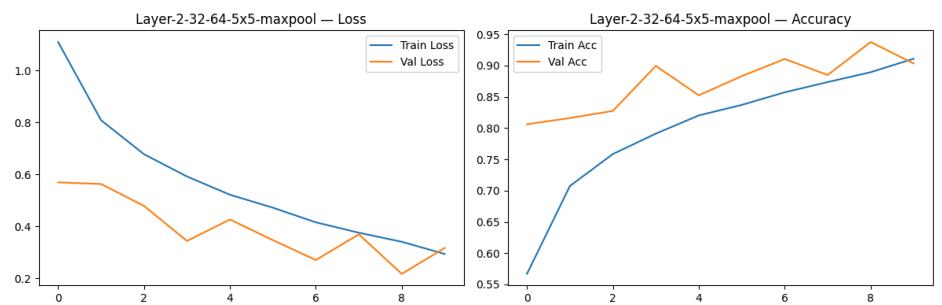
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7849

Penjelasan:

Pooling layer digunakan untuk melakukan downsampling pada feature maps dengan tujuan mengurangi dimensi spasial. Max pooling akan mengambil nilai terbesar dari suatu area atau *window*, sedangkan Average pooling akan mengambil nilai rata-ratanya. Secara teori, max pooling mempertahankan fitur yang paling aktif atau paling dominan. Max pooling akan cocok jika keberadaan suatu fitur lebih penting daripada distribusi detailnya. Contohnya jika suatu filter mendeteksi edge kuat, max pooling akan mempertahankan respons edge tersebut. Tetapi kekurangannya adalah max pooling dapat membuang informasi detail non-dominan. Berbeda dengan average pooling yang mempertahankan informasi distribusi umum dari suatu area atau *window*.

Dari eksperimen ini, max pooling terbukti lebih unggul dibandingkan average pooling. Operasi agregasi nilai rata-rata dari average pooling ini memberikan efek *smoothing* pada feature maps yang dapat memburamkan fitur. Berbeda dengan max pooling yang justru menjadi penyeleksi sinyal terkuat dan secara agresif mempertahankan dan meneruskan fitur berfrekuensi tinggi yang mencolok seperti edge dan tekstur kasar. Dan grafik dari Max Pooling menunjukkan penurunan loss yang lebih stabil daripada ketika menggunakan Average Pooling.

2.2.1.5 Analisis LocallyConnected2D Scratch (Shared vs Non-Shared)

Notebook Pengujian:

- **Notebook 1:** Layer-2-32-64-5x5-maxpool.ipynb
- **Notebook 2:** locally_connected_scratch.ipynb

No	Hasil Pengujian
1	Layer-2-32-64-5x5-maxpool.ipynb: keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid', name='conv2s_2'), keras.layers.MaxPooling2D((2, 2), name='maxpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'),

```
keras.layers.Dense(6, activation='softmax', name='output'),
```

Parameter:

Model: "Layer-2-32-64-5x5-maxpool"

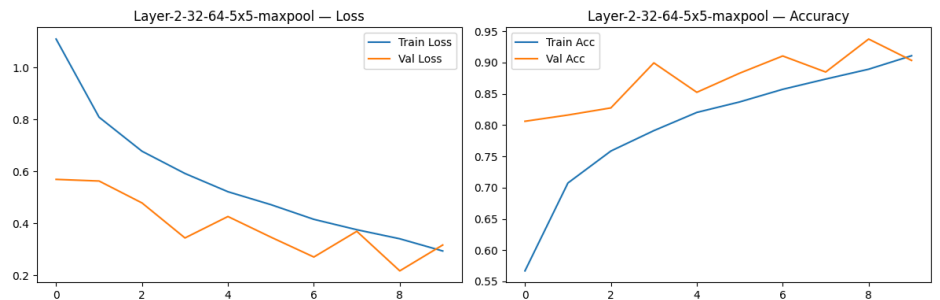
Layer (type)	Output Shape	Param #
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0
flatten (Flatten)	(None, 73984)	0
dense_1 (Dense)	(None, 128)	9,470,080
output (Dense)	(None, 6)	774

Total params: 9,524,550 (36.33 MB)

Trainable params: 9,524,550 (36.33 MB)

Non-trainable params: 0 (0.00 B)

Plot:



Macro F1 Score: 0.7845

Waktu: 45.04s

2

locally_connected_scratch.ipynb:

```
conv2s_1 (Conv2D) -> LocallyConnected2DLayer: kernel (5, 5, 3, 32) tiled to (21316, 75, 32) | (150, 150, 3) -> (146, 146, 32)
maxpool_1 (MaxPooling2D) -> MaxPooling2DLayer: (146, 146, 32) -> (73, 73, 32)
conv2s_2 (Conv2D) -> LocallyConnected2DLayer: kernel (5, 5, 32, 64) tiled to (4761, 800, 64) | (73, 73, 32) -> (69, 69, 64)
maxpool_2 (MaxPooling2D) -> MaxPooling2DLayer: (69, 69, 64) -> (34, 34, 64)
flatten (Flatten) -> FlattenLayer: (34, 34, 64) -> (73984,)
dense_1 (Dense) -> Dense(73984, 128) + ActivationLayer(relu)
output (Dense) -> Dense(128, 6) + ActivationLayer(softmax)
```

Parameters:	
<pre> Layer Type Param # ===== LocallyConnected2DLayer 51,840,512 MaxPooling2DLayer 0 LocallyConnected2DLayer 244,067,904 MaxPooling2DLayer 0 FlattenLayer 0 Dense 9,470,080 ActivationLayer 0 Dense 774 ActivationLayer 0 ===== Total Trainable Params: 305,379,270 </pre>	
Macro F1 Score: 0.7838 Waktu : 4267.80s	

Penjelasan:

LocallyConnected2D memakai unshared weights, dimana setiap posisi spasial punya filter sendiri, sedangkan Conv2D memiliki shared weights, yang dimana filternya sama untuk semua posisi spasial. Perbedaan ini umumnya berdampak terhadap jumlah parameter, generalisasi, dan efisiensi komputasi. Akibat dari LocallyConnected2D yang setiap posisi output punya filter sendiri, model dapat belajar pola yang berbeda untuk lokasi yang berbeda. Ini membuat ledakan jumlah parameter dari Conv2D ke LocallyConnected2D. Jika dilihat dari hasil eksperimen, total trainable params meningkat berkali-kali lipat dari 9,524,550 -> 305,379,270 dan ini sangat berat secara komputasi.

Sebagai informasi tambahan, proses inference saat evaluasi keras kembali adalah sekitar 45 detik, sedangkan di LocallyConnected2d layer mencapai 4267.80s (~1,18 jam). Namun, dengan komputasi yang sudah seberat itu, nilai Macro F1 Score justru menurun sedikit sebesar 0.000674 poin.

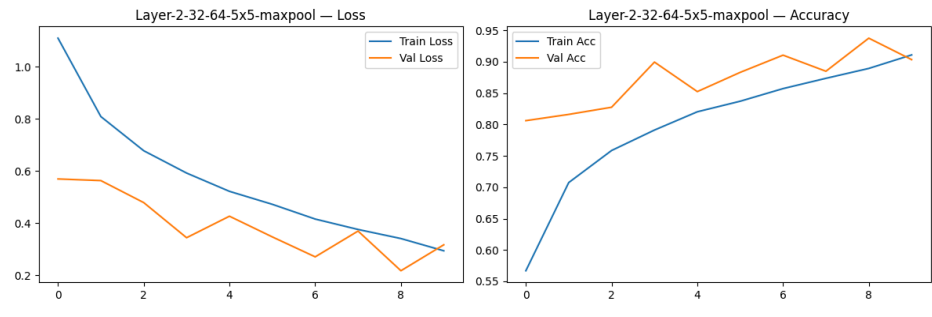
2.2.1.6 Analisis Conv2D Scratch

Notebook Pengujian:

- **Notebook 1:** Layer-2-64-128-5x5-maxpool.ipynb

- **Notebook 2:** scratch_vs_keras.ipynb

No	Hasil Pengujian																								
1	Layer-2-32-64-5x5-maxpool.ipynb: <pre>keras.layers.Input(shape=IMG_SIZE + (3,), name='input'), keras.layers.Conv2D(32, (5, 5), activation='relu', padding='valid', name='conv2s_1'), keras.layers.MaxPooling2D((2, 2), name='maxpool_1'), keras.layers.Conv2D(64, (5, 5), activation='relu', padding='valid', name='conv2s_2'), keras.layers.MaxPooling2D((2, 2), name='maxpool_2'), keras.layers.Flatten(name='flatten'), keras.layers.Dense(128, activation='relu', name='dense_1'), keras.layers.Dense(6, activation='softmax', name='output'),</pre> Parameter: Model: "Layer-2-32-64-5x5-maxpool" <table><thead><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr></thead><tbody><tr><td>conv2s_1 (Conv2D)</td><td>(None, 146, 146, 32)</td><td>2,432</td></tr><tr><td>maxpool_1 (MaxPooling2D)</td><td>(None, 73, 73, 32)</td><td>0</td></tr><tr><td>conv2s_2 (Conv2D)</td><td>(None, 69, 69, 64)</td><td>51,264</td></tr><tr><td>maxpool_2 (MaxPooling2D)</td><td>(None, 34, 34, 64)</td><td>0</td></tr><tr><td>flatten (Flatten)</td><td>(None, 73984)</td><td>0</td></tr><tr><td>dense_1 (Dense)</td><td>(None, 128)</td><td>9,470,080</td></tr><tr><td>output (Dense)</td><td>(None, 6)</td><td>774</td></tr></tbody></table> Total params: 9,524,550 (36.33 MB) Trainable params: 9,524,550 (36.33 MB) Non-trainable params: 0 (0.00 B) Plot:	Layer (type)	Output Shape	Param #	conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432	maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0	conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264	maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0	flatten (Flatten)	(None, 73984)	0	dense_1 (Dense)	(None, 128)	9,470,080	output (Dense)	(None, 6)	774
Layer (type)	Output Shape	Param #																							
conv2s_1 (Conv2D)	(None, 146, 146, 32)	2,432																							
maxpool_1 (MaxPooling2D)	(None, 73, 73, 32)	0																							
conv2s_2 (Conv2D)	(None, 69, 69, 64)	51,264																							
maxpool_2 (MaxPooling2D)	(None, 34, 34, 64)	0																							
flatten (Flatten)	(None, 73984)	0																							
dense_1 (Dense)	(None, 128)	9,470,080																							
output (Dense)	(None, 6)	774																							

	 Macro F1 Score: 0.7839 Waktu: 16.38s
2	Macro F1 Score: 0.7838 Waktu: 1900.95s

Penjelasan:

Ketika evaluasi dari keras dan scratch dibandingkan, tidak terdapat perbedaan yang signifikan terkait Macro F1 Score dan perbedaannya hanya di sekitar 0.000056 poin, namun waktu evaluasi yang meningkat drastis.

2.2.2 RNN

2.2.2.1 Pengaruh Jumlah Layer

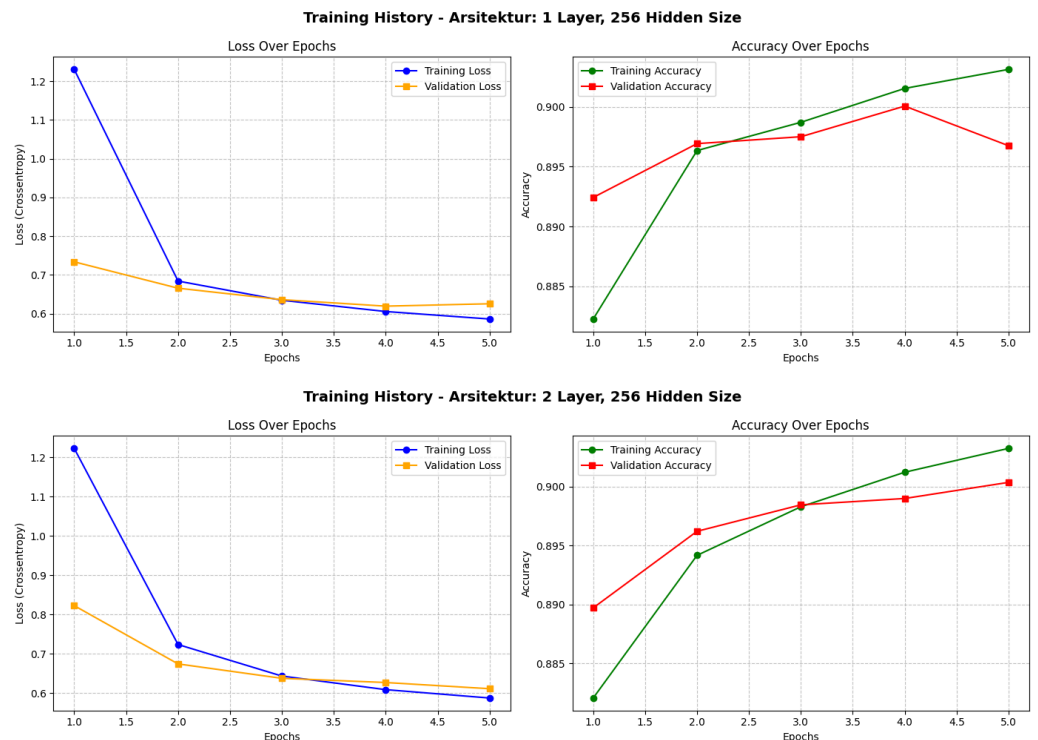
Jumlah Layer	Hidden Size	BLEU-4	METEOR	Waktu Training (s)
1	256	0.0820	0.2123	2930.94
2		0.0817	0.1933	3599.92
3		0.0184	0.1759	4326.59

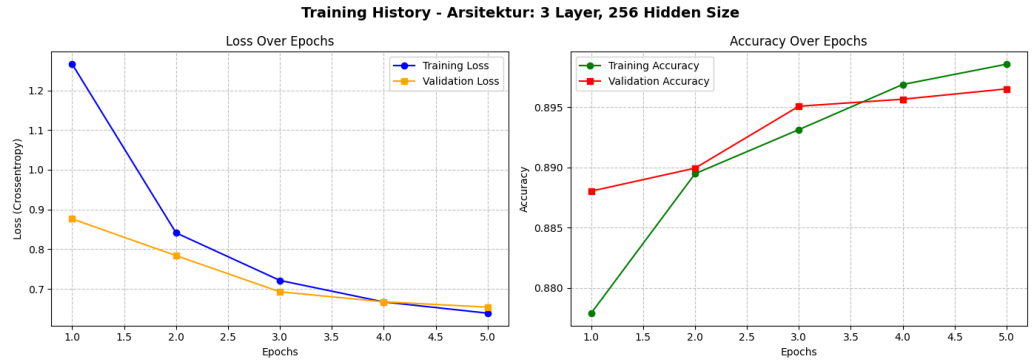
Penjelasan:

Berdasarkan hasil eksperimen, penambahan jumlah layer recurrent pada decoder RNN tidak selalu memberikan peningkatan performa. Model dengan 1 layer menghasilkan BLEU-4 sebesar 0.0820 dengan METEOR 0.2123. Ketika jumlah layer ditambah menjadi 2, BLEU-4 sedikit turun menjadi 0.0817 sementara METEOR juga menurun

ke 0.1933, meskipun perbedaannya masih sangat kecil. Penurunan signifikan terjadi ketika layer ditambah menjadi 3, di mana BLEU-4 anjlok drastis ke 0.0184 dan METEOR turun ke 0.1759.

Fenomena ini konsisten dengan masalah **vanishing gradient** yang menjadi kelemahan utama arsitektur SimpleRNN. Pada RNN standar, gradien yang dipropagasikan mundur melalui banyak timestep cenderung mengecil secara eksponensial. Ketika jumlah layer recurrent ditambah, jalur propagasi gradien semakin panjang dan dalam, memperparah masalah ini sehingga bobot pada layer-layer awal sulit diperbarui secara efektif. Akibatnya model dengan 3 layer justru gagal belajar representasi yang bermakna. Selain itu, waktu training meningkat signifikan seiring penambahan layer, dari 2930.94 detik untuk 1 layer hingga 4326.59 detik untuk 3 layer, tanpa disertai peningkatan performa.





Dari grafik training dan validation loss, model dengan 1 layer menunjukkan kurva konvergensi yang paling stabil. Model dengan 3 layer memperlihatkan validation loss yang stagnan dan tidak menurun, mengindikasikan bahwa model gagal melakukan generalisasi.

2.2.2.2 Pengaruh *Hidden Size*

Jumlah Layer	Hidden Size	BLEU-4	METEOR	Waktu Training (s)
1	256	0.0820	0.2123	2930.94
	512	0.1075	0.1933	4506.31

Penjelasan:

Peningkatan hidden size dari 256 menjadi 512 memberikan dampak positif yang lebih jelas dibandingkan penambahan jumlah layer. Model dengan hidden size 512 menghasilkan BLEU-4 sebesar 0.1075, meningkat sekitar 31% dibandingkan hidden size 256 yang hanya menghasilkan 0.0820. Namun METEOR justru turun dari 0.2123 ke 0.1933, yang menunjukkan bahwa peningkatan BLEU-4 tidak sepenuhnya sejalan dengan kualitas caption secara keseluruhan.

Hidden size yang lebih besar memberikan kapasitas representasi yang lebih luas pada setiap timestep. Dengan dimensi hidden state 512, model mampu menyimpan lebih banyak informasi konteks dari timestep sebelumnya dalam satu vektor, sehingga prediksi kata berikutnya lebih akurat secara n-gram (yang diukur BLEU-4). Trade-off yang muncul

adalah waktu training yang meningkat dari 2930.94 detik menjadi 4506.31 detik, hampir 54% lebih lama.



2.2.2.3 Perbandingan Keras

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
Scratch	1	256	0.0841	0.2191	215.41
Keras			0.0841	0.2191	589.97

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
Scratch	1	512	0.1059	0.2258	543.71
Keras			0.1059	0.2258	518.46

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	2	256	0.0823	0.2002	176.42
Keras			0.0823	0.2002	439.90

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	2	512	0.1039	0.2261	603.73
Keras			0.1039	0.2261	521.25

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	3	256	0.0257	0.1770	192.13
Keras			0.0257	0.1770	485.83

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	3	512	0.0801	0.2133	659.48
Keras			0.0801	0.2133	572.46

Penjelasan:

Hasil perbandingan menunjukkan bahwa implementasi from scratch menghasilkan nilai BLEU-4 dan METEOR yang identik dengan implementasi Keras untuk seluruh 6 variasi. Hal ini memvalidasi kebenaran implementasi *forward propagation* dari scratch bobot yang sama menghasilkan output yang sama persis, tanpa perbedaan numerik yang berarti.

Perbedaan yang signifikan justru terletak pada waktu inferensi. Implementasi Keras secara konsisten lebih cepat untuk hidden size 256

(misalnya 589.97 detik vs 215.41 detik untuk L=1, H=256), namun untuk hidden size 512 hasilnya bervariasi pada beberapa konfigurasi scratch bahkan lebih cepat dari Keras. Hal ini disebabkan karena overhead Keras yang mencakup graph compilation dan session management memberikan latensi tambahan yang lebih terasa pada model kecil. Sedangkan untuk model yang lebih besar (hidden size 512), efisiensi operasi batch NumPy pada implementasi scratch dapat bersaing dengan Keras karena sebagian besar komputasi sudah tervektorisasi pada level operasi matriks.

2.2.3 LSTM

2.2.3.1 Pengaruh Jumlah Layer

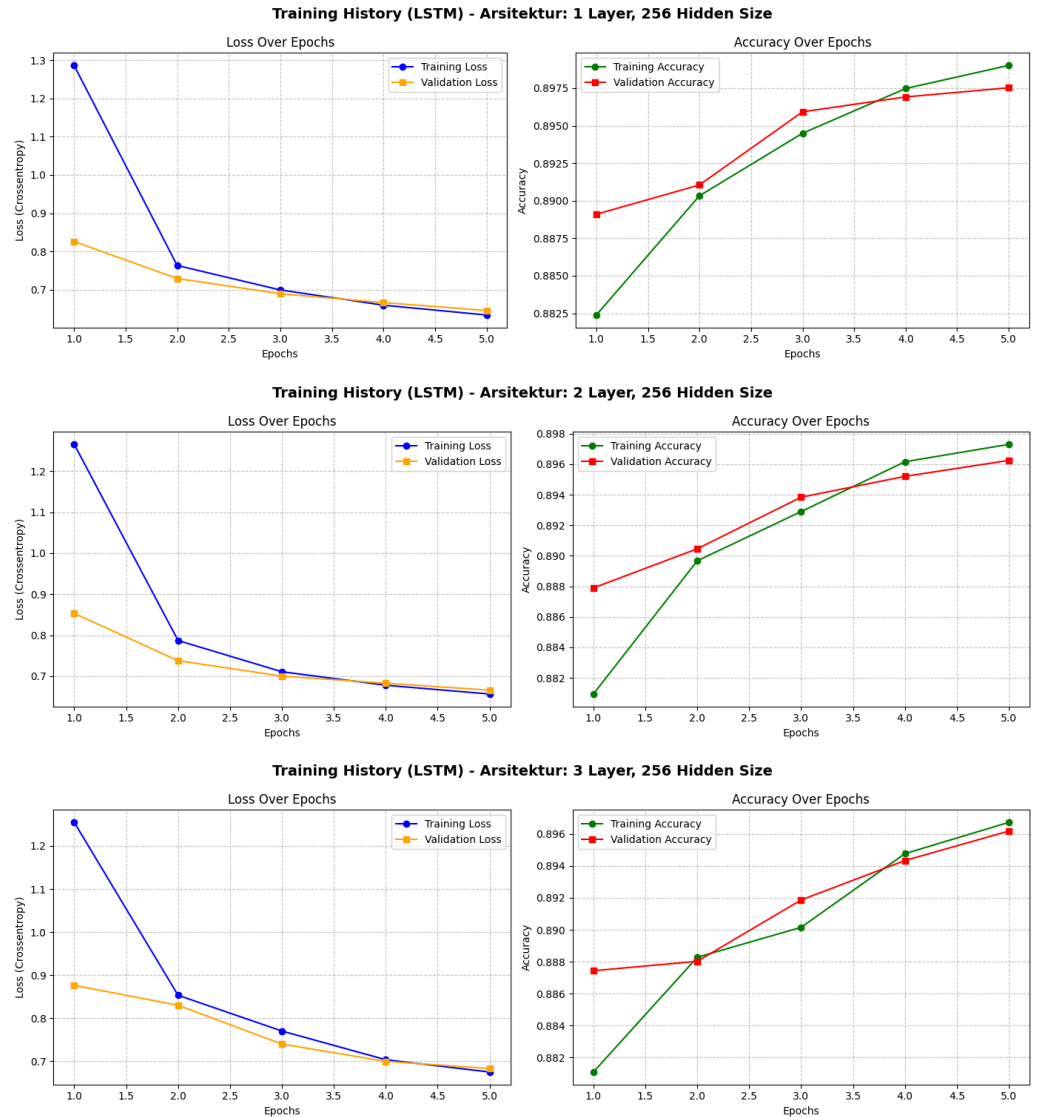
Jumlah Layer	Hidden Size	BLEU-4	METEOR	Waktu Training (s)
1	256	0.0407	0.2047	3391.22
2		0.0407	0.2047	3471.14
3		0.0817	0.1933	5408.29

Penjelasan:

Berbeda dengan pola yang terlihat pada RNN, model LSTM menunjukkan tren yang menarik terhadap penambahan jumlah layer. Model dengan 1 dan 2 layer menghasilkan BLEU-4 yang identik yaitu 0.0407 dengan METEOR 0.2047, sedangkan model dengan 3 layer justru meningkat ke BLEU-4 0.0817 dan METEOR 0.1933.

Peningkatan performa pada 3 layer LSTM ini dapat dijelaskan oleh mekanisme gating yang dimiliki LSTM. Tidak seperti RNN yang rentan terhadap vanishing gradient, LSTM memiliki cell state dan forget gate yang memungkinkan gradien mengalir lebih stabil melalui banyak timestep maupun layer yang lebih dalam. Akibatnya, penambahan layer pada LSTM tidak langsung menyebabkan degradasi performa seperti yang terjadi pada RNN. Model LSTM yang lebih dalam justru dapat membangun representasi hierarkis yang lebih kaya layer pertama mempelajari pola sintaksis dasar, sementara layer berikutnya menangkap hubungan semantik yang lebih kompleks.

Namun demikian, waktu training meningkat cukup signifikan, dari 3391.22 detik untuk 1 layer hingga 5408.29 detik untuk 3 layer. Ini adalah konsekuensi langsung dari penambahan operasi perhitungan 4 gate per layer per timestep.



2.2.3.2 Pengaruh *Hidden Size*

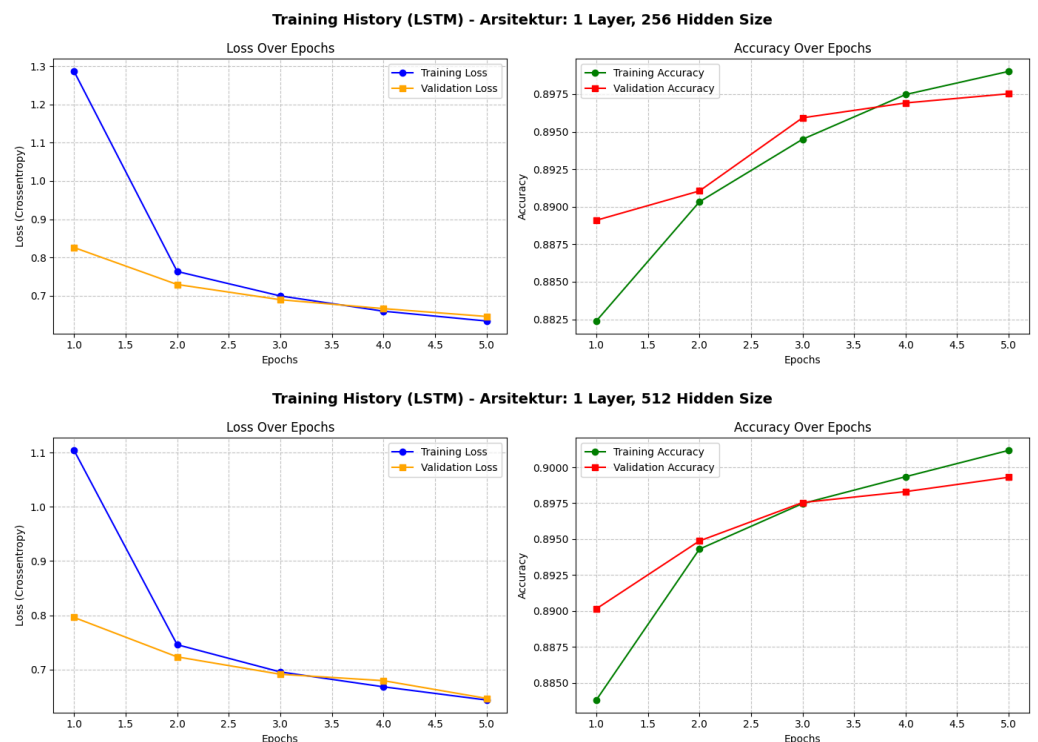
Jumlah Layer	Hidden Size	BLEU-4	METEOR	Waktu Training (s)
1	256	0.0407	0.2047	2930.94

	512	0.0816	0.1932	4506.31
--	-----	--------	--------	---------

Penjelasan:

Peningkatan hidden size dari 256 ke 512 pada LSTM memberikan dampak yang signifikan terhadap BLEU-4, meningkat dari 0.0407 menjadi 0.0816 hampir dua kali lipat. Namun METEOR sedikit turun dari 0.2047 ke 0.1932, pola yang sama dengan yang diamati pada RNN.

Kapasitas hidden state yang lebih besar memungkinkan LSTM menyimpan informasi cell state yang lebih kaya, sehingga keempat gate dapat beroperasi pada ruang representasi yang lebih ekspresif. Hal ini sangat membantu dalam task image captioning di mana model perlu mempertahankan konteks visual dari feature gambar sekaligus membangun urutan kata yang koheren. Trade-off yang terjadi adalah waktu training yang hampir dua kali lebih lama, dari 2930.94 detik menjadi 4506.31 detik.



2.2.3.3 Perbandingan Keras

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	1	256	0.0378	0.2120	793.03
Keras			0.0378	0.2120	585.33

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	1	512	0.0823	0.2001	2753.93
Keras			0.0823	0.2001	644.60

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	2	256	0.0378	0.2120	922.19
Keras			0.0378	0.2120	586.18

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	2	512	0.0378	0.2120	2755.92
Keras			0.0378	0.2120	795.31

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
<i>Scratch</i>	3	256	0.0823	0.2002	732.54
Keras			0.0823	0.2002	591.94

Model	Layer	Hidden Size	BLEU-4	METEOR	Waktu Inferensi (s)
Scratch	3	512	0.0598	0.1655	4749.69
Keras			0.0598	0.1655	1738.10

Penjelasan:

Serupa dengan hasil pada RNN, implementasi LSTM from scratch menghasilkan nilai BLEU-4 dan METEOR yang identik dengan Keras pada seluruh 6 variasi. Ini mengonfirmasi bahwa implementasi keempat gate LSTM (input, forget, cell candidate, output) beserta mekanisme cell state dan hidden state telah diimplementasikan dengan benar.

Perbedaan yang paling mencolok adalah waktu inferensi. Untuk hidden size 512, scratch secara konsisten jauh lebih lambat dari Keras, misalnya pada L=1 H=512, scratch memerlukan 2753.93 detik sementara Keras hanya 644.60 detik, selisih lebih dari 4 kali lipat. Perbedaan ini jauh lebih besar dibandingkan RNN karena setiap timestep LSTM melakukan 4 kali lebih banyak operasi matriks dibandingkan RNN (satu per gate), dan semua operasi tersebut dieksekusi dalam loop Python pada implementasi scratch. Keras di sisi lain mengeksekusi seluruh operasi gate tersebut sebagai satu operasi CUDA yang terfusi, sehingga overhead per-timestep jauh lebih kecil.

2.2.4 RNN & LSTM

Jumlah Layer	Hidden Size	BLEU-4 RNN	BLEU-4 LSTM	METEOR RNN	METEOR LSTM	Waktu Training (s)	
						RNN	LSTM
1	256	0.0820	0.0407	0.2123	0.2047	2930.94	3391.22
1	512	0.1075	0.0817	0.2195	0.1933	4506.31	5070.34
2	256	0.0817	0.0407	0.1933	0.2047	3599.92	3471.14
2	512	0.1002	0.0407	0.2170	0.2045	5464.30	4009.09
3	256	0.0184	0.0817	0.1759	0.1933	4326.59	5408.29
3	512	0.0781	0.0594	0.2056	0.1623	6317.05	5292.38

Penjelasan:

Secara keseluruhan, RNN mengungguli LSTM pada mayoritas konfigurasi berdasarkan BLEU-4. Dari 6 variasi yang diuji, RNN menghasilkan BLEU-4 lebih tinggi pada 5 dari 6 konfigurasi satu-satunya pengecualian adalah konfigurasi 3 layer hidden 256 di mana LSTM (0.0817) lebih baik dari RNN (0.0184). Nilai BLEU-4 terbaik secara keseluruhan dicapai oleh RNN dengan 1 layer hidden size 512 (0.1075), sementara BLEU-4 terbaik LSTM adalah 0.0817 pada konfigurasi 3 layer hidden 256.

Hasil ini mungkin terlihat kontra-intuitif mengingat LSTM secara teoritis lebih *powerful*. Namun hal ini dapat dijelaskan oleh beberapa faktor. Pertama, dengan hanya 5 epoch training pada dataset yang relatif kecil (6000 gambar), LSTM yang memiliki jauh lebih banyak parameter (4x parameter recurrent dibanding RNN) membutuhkan lebih banyak iterasi untuk konvergen. RNN yang lebih sederhana justru lebih cepat mencapai solusi yang cukup baik dalam jumlah epoch yang terbatas. Kedua, caption pada dataset Flickr8k umumnya pendek (rata-rata 10-15 kata), sehingga keunggulan LSTM dalam menangani dependensi jangka panjang tidak terlalu dimanfaatkan secara maksimal.

Dari sisi METEOR, pola lebih bervariasi LSTM kadang menghasilkan METEOR lebih tinggi meski BLEU-4-nya lebih rendah, mengindikasikan bahwa caption yang dihasilkan LSTM memiliki overlap semantik yang lebih baik dengan referensi meski tidak unggul secara presisi n-gram. Waktu training LSTM secara konsisten lebih lama dari RNN pada konfigurasi yang setara, rata-rata sekitar 15-20% lebih lambat, karena beban komputasi keempat gate per timestep.

2.2.5 Pengaruh *Max Caption Length*

Adapun *best* model didapatkan untuk pengujian:

RNN & LSTM: 1 layer, 512 hidden size

Arsitektur	Max-Length	BLEU-4	Waktu Inferensi (s)
RNN	3	0.0234	2.66
	5	0.0974	3.16

	10	0.0973	5.92
LSTM	3	0.0172	3.54
	5	0.0769	10.91
	10	0.0763	30.85

Penjelasan:

Eksperimen dilakukan menggunakan best model yaitu RNN dan LSTM dengan 1 layer hidden size 512, dengan memvariasikan panjang maksimum caption yang dihasilkan saat inferensi (3, 5, dan 10 token). Pada RNN, BLEU-4 meningkat drastis dari 0.0234 pada max length 3 menjadi 0.0974 pada max length 5, kemudian hampir stagnan di 0.0973 pada max length 10. Pola serupa terjadi pada LSTM BLEU-4 naik dari 0.0172 pada max length 3 menjadi 0.0769 pada max length 5, lalu sedikit turun ke 0.0763 pada max length 10.

Max length 3 menghasilkan BLEU-4 yang sangat rendah karena caption yang dihasilkan terlalu pendek untuk melewati threshold n-gram order tinggi (trigram dan 4-gram) yang dibutuhkan BLEU-4. Caption 3 kata hampir tidak memiliki 4-gram yang bisa dicocokkan dengan referensi, sehingga precision turun drastis. Peningkatan ke max length 5 signifikan karena sudah cukup untuk menghasilkan kalimat pendek yang bermakna. Setelah itu, BLEU-4 cenderung stagnan karena model secara alami sudah mengeluarkan token <end> sebelum mencapai batas panjang artinya max length 10 tidak berbeda jauh dengan 5 dalam praktiknya karena caption yang dihasilkan memang rata-rata hanya 5-8 kata.

Waktu inferensi meningkat seiring max length yang lebih panjang, dan LSTM secara konsisten lebih lambat dari RNN pada setiap konfigurasi max length yang sama karena komputasi gate yang lebih banyak per timestep.

BAB III: Kesimpulan dan Saran

3.1 Kesimpulan

Berdasarkan hasil implementasi dan eksperimen yang telah dilakukan, dapat diambil kesimpulan bahwa forward propagation untuk ketiga arsitektur, CNN, RNN, dan LSTM dapat berjalan sebagaimana mestinya dibuktikan dengan hasil perbandingan dengan model Keras yang serupa. Jumlah layer konvolusi, banyak dan ukuran filter per konvolusi, dan jenis pooling memiliki pengaruh yang cukup signifikan pada arsitektur CNN. Adapun performa arsitektur RNN dan LSTM dipengaruhi oleh jumlah layer dan ukuran hidden statenya. Pada pelatihan dengan epoch singkat, hasil pengujian menunjukkan bahwa RNN lebih unggul dari LSTM pada mayoritas konfigurasi. Secara garis besar, implementasi dan eksperimen ini berhasil mengaplikasikan CNN, RNN, dan LSTM dan menganalisis pengaruh-pengaruh konfigurasi dan arsitekturnya.

3.2 Saran

1. Setup untuk TensorFlow seharusnya dilakukan lebih awal.
2. Dapat dilakukan eksperimen lanjutan untuk CNN yaitu memodifikasi spasial minor pada beberapa gambar dan membandingkannya dengan gambar original untuk membandingkan hasil Conv2D (Shared) dan LocallyConnected2D (Non-Shared) lebih detail dan membuktikan konsep *Translation Invariance*.
3. Sebaiknya epoch pelatihan untuk RNN dan LSTM dibuat sekitar 50-100 agar dapat menghasilkan perbandingan yang lebih optimal, disertai dengan pipeline Tensorflow yang dijalankan berbasis GPU agar eksekusi dapat lebih cepat dan menghemat waktu.

LAMPIRAN

Link Repository: <https://github.com/ivant8k/CNN-RNN-LSTM-Scratch>

Tabel 4.1 Pembagian tugas anggota

NIM	Pembagian Tugas
13523129	Implementasi CNN dan Laporan
13523144	Implementasi RNN & LSTM dan Laporan
13523164	Implementasi Pengujian RNN & LSTM dan Laporan

REFERENSI

NumPy Developers. (n.d.). numpy.einsum. NumPy. Retrieved May 9, 2026, from <https://numpy.org/doc/stable/reference/generated/numpy.einsum.html>

Scherer, D., Müller, A., & Behnke, S. (2010). Evaluation of pooling operations in convolutional architectures for object recognition. In K. Diamantaras, W. Duch, & L. S. Iliadis (Eds.), Artificial neural networks – ICANN 2010: 20th International Conference, Thessaloniki, Greece, September 15–18, 2010, proceedings, part III (pp. 92–101). Springer. https://doi.org/10.1007/978-3-642-15825-4_10

Keras. (n.d.). Conv2D layer. Retrieved May 9, 2026, from https://keras.io/api/layers/convolution_layers/convolution2d/

Keras. (n.d.). Pooling layers. Retrieved May 9, 2026, from https://keras.io/api/layers/pooling_layers/

Keras. (n.d.). Flatten layer. Retrieved May 9, 2026, from https://keras.io/api/layers/reshaping_layers/flatten/

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (n.d.). Convolutions for images. In Dive into deep learning. Retrieved May 10, 2026, from https://d2l.ai/chapter_convolutional-neural-networks/conv-layer.html

Stanford University. (n.d.). Convolutional neural networks. CS231n: Deep learning for computer vision. Retrieved May 10, 2026, from <https://cs231n.github.io/convolutional-networks/>

Lampiran Form Penggunaan AI

Dengan ini, kami:

Nama/NIM : Ivant/13523129, Nazih/13523144, Rizain/13523164
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang kami gunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan	Tidak	-
Pembuatan Teks	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik	Tidak	-
Penasehat terkait alur skema proses implementasi <i>source code</i>	Ya	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/Kecerdasan buatan yang dilakukan pada matakuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 15 Mei 2026



Ivant Samuel Silaban
13523129

Jakarta, 15 Mei 2026



Muhamad Nazih Najmudin
13523144

Jakarta, 15 Mei 2026



Muhammad Rizain Firdaus
13523164